P NEEHARIKA-2520090233

PRACTICAL— SESSION 11

Task 1 :- Develop a multithreaded counter application using POSIX threads (`pthread_create`, `pthread_join`). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

```
  GNU nano 7.2                  task1_race_condition.c *
#include <stdio.h>
#include <pthread.h>

#define THREADS 4
#define ITERATIONS 1000000

long long counter = 0;

void *increment(void *arg) {
    for (int i = 0; i < ITERATIONS; i++) {
        counter++;
    }

    return NULL;
}

int main() {
    pthread_t threads[THREADS];

    printf("Multithreaded Counter Demonstration\n");
    printf("Expected counter: %d\n",
           THREADS * ITERATIONS);

    for (int i = 0; i < THREADS; i++) {
        pthread_create(&threads[i], NULL, increment, NULL);
    }

    for (int i = 0; i < THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Actual counter: %lld\n", counter);
    printf("All threads completed.\n");

    return 0;
}
root@HP:~/practical_session11# gcc -pthread task1_race_condition.c -o task1_
race_condition
root@HP:~/practical_session11# ./task1_race_condition
Multithreaded Counter Demonstration
Expected counter: 4000000
Actual counter: 1353733
All threads completed.
root@HP:~/practical_session11#
```

Task 2 :- Modify the previous program using mutex locks (`pthread_mutex_lock`, `pthread_mutex_unlock`) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

```
  GNU nano 7.2                    task2_mutex_counter.c *
#include <stdio.h>
#include <pthread.h>

#define THREADS 4
#define ITERATIONS 1000000

long long unsafe_counter = 0;
long long safe_counter = 0;

pthread_mutex_t lock;

void *increment(void *arg) {
    for (int i = 0; i < ITERATIONS; i++) {
        unsafe_counter++;
    }

    for (int i = 0; i < ITERATIONS; i++) {
        pthread_mutex_lock(&lock);
        safe_counter++;
        pthread_mutex_unlock(&lock);
    }

    return NULL;
}

int main() {
    pthread_t threads[THREADS];

    pthread_mutex_init(&lock, NULL);
```

```c
    printf("Mutex Synchronization Demonstration\n");
    printf("Expected counter: %d\n",
           THREADS * ITERATIONS);

    for (int i = 0; i < THREADS; i++) {
        pthread_create(&threads[i], NULL, increment, NULL);
    }

    for (int i = 0; i < THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Counter without mutex: %lld\n", unsafe_counter);
    printf("Counter with mutex: %lld\n", safe_counter);

    pthread_mutex_destroy(&lock);

    printf("All threads completed successfully.\n");

    return 0;
}
```

Output Description :- The output compares the counter values with and without mutex locks. The synchronized counter reaches the expected value of 4,000,000.

```
root@HP:~/practical_session11# gcc -pthread task2_mutex_counter.c -o task2_m
utex_counter
root@HP:~/practical_session11# ./task2_mutex_counter
Mutex Synchronization Demonstration
Expected counter: 4000000
Counter without mutex: 1432648
Counter with mutex: 4000000
All threads completed successfully.
root@HP:~/practical_session11#
```

Observation :- Multithreading and race conditions were demonstrated successfully, and mutex locks were used to synchronize access to a shared variable.